

**SIMATS ENGINEERING
ASSESSMENT TOOL 2**

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 50%

Duration: 1 Hour

QUESTIONS: 3

Total Marks:30

Annexure A

Dry Run Evaluation

Code of Conduct:

I Konkala Reddi Kishore Reddy(192312523)
(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Dry Run Evaluation

1.Scenario

A board-game company is developing an AI opponent for a turn-based strategy game similar to checkers, to be deployed on low-power tablets. The AI must compete against human players, make decisions within strict time limits, evaluate complex board states with multiple possible moves, and the decision tree is extremely large, making computation expensive on limited hardware.

Tasks:

- i. Explain how the Minimax algorithm models this problem.
- ii. Analyze the computational challenges of large game trees.
- iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning.
- iv. Design an evaluation function and justify the factors considered.
- v. Recommend a strategy for balancing optimality and real-time performance.

2. Scenario

A city's emergency dispatch center uses an AI routing system to send ambulances to accident sites during peak-hour traffic. The environment is highly dynamic - roads may suddenly close due to accidents or construction, and travel cost varies with congestion. The dispatch system has access to: a heuristic estimate (straight-line distance to the incident), partial real-time updates about road closures, and variable travel costs depending on traffic and time of day. Because every second counts, the system must quickly decide the ambulance's route with minimum delay and risk.

Tasks:

- i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty.
- ii. Explain how A* would handle the same situation, including how it balances cost and heuristic.
- iii. Analyze the impact of heuristic accuracy on both algorithms.
- iv. Discuss how dynamic changes (sudden road closures) affect search performance.
- v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety.

3. Scenario

An airline implements an AI system to optimize crew scheduling across hundreds of flights and thousands of crew members. The objective is to minimize crew costs, layover time, and scheduling conflicts while satisfying regulatory rest requirements. The system must continuously adjust assignments as flight delays occur.

However, the search space is extremely large, flight schedules change dynamically, and the system may get stuck in locally optimal crew assignments. Tasks: Formulate this as an optimization problem and explain why traditional search is inefficient.

- i. Explain how Hill Climbing would work in this scenario and identify its limitations.
- ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation.
- iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations.
- iv. Recommend the best approach and justify your choice based on scalability and adaptability.

1. AI Opponent for a Turn-Based Strategy Game

i. How Minimax models the problem

Minimax is suitable for a two-player, turn-based, zero-sum game where one player's gain is the other player's loss.

The game can be represented as a game tree:

- Root node: Current board position.
- MAX nodes: AI's turns; AI selects the move with the highest evaluation.
- MIN nodes: Human player's turns; AI assumes the opponent chooses the move that is worst for the AI.
- Leaf nodes: Final or depth-limited board positions.
- Evaluation function: Assigns a numerical value to each leaf.

The AI evaluates possible future moves and chooses the move that maximizes its guaranteed outcome, assuming the opponent plays optimally.

ii. Computational challenges of large game trees

If b is the average number of legal moves and d is the search depth, Minimax has approximately $O(b^d)$ time complexity.

For example, if a game has 20 possible moves per turn and the AI searches 8 levels, $20^8 = 25.6 \times 10^{10}$ possible nodes.

Major challenges are:

1. High computational cost.
2. High memory requirements.
3. Strict response-time limits.
4. Limited CPU, memory, and battery resources on tablets.
5. Expensive board-state evaluation.

Therefore, complete Minimax search is usually impractical.

iii. Alpha-Beta pruning

Alpha-Beta pruning eliminates branches that cannot influence the final Minimax decision.

Alpha (α) is the best value MAX can guarantee so far. Beta (β) is the best value MIN can guarantee so far. Pruning occurs when $\alpha \geq \beta$.

Alpha-Beta pruning produces the same result as Minimax while searching fewer nodes. With good move ordering, the theoretical complexity can improve from $O(b^d)$ to approximately $O(b^{(d/2)})$.

Benefits include reduced computation, deeper search within the same time, lower CPU usage, and better suitability for low-power devices.

iv. Evaluation function

A suitable evaluation function is:

$$E(s) = w_1M + w_2P + w_3K + w_4C + w_5D$$

where:

- M = material/piece advantage
- P = positional advantage
- K = king or special-piece advantage
- C = control of important board areas
- D = defensive/safety advantage
- $w_1 \dots w_5$ = weights

Material measures the number and value of pieces. Position considers strategically important squares. Special pieces may have greater mobility. Board control indicates strategic influence. Safety measures vulnerability to attacks.

The weights should be tuned using simulated games and testing against human players.

v. Strategy for balancing optimality and real-time performance

The recommended strategy is iterative-deepening Minimax with Alpha-Beta pruning.

The AI searches progressively deeper:

Depth 1 → Depth 2 → Depth 3 → Depth 4 → ...

If the time limit expires, it returns the best move from the last completed search depth.

Additional techniques:

1. Alpha-Beta pruning
2. Iterative deepening
3. Good move ordering
4. Transposition tables
5. Depth limits
6. Efficient board representation
7. Time-based search cutoff
8. Extensions for tactically important positions

This provides fast, strong, and reliable decisions without requiring exhaustive search.

2. Emergency Ambulance Routing

i. Greedy Best-First Search

Greedy Best-First Search selects the node with the smallest heuristic:

$$f(n) = h(n)$$

where $h(n)$ is the estimated distance from the current location to the accident.

It is very fast and can quickly find a route. However, it ignores the actual cost already travelled, $g(n)$. Therefore, it may choose a geographically short road that has heavy traffic, construction, closures, or high risk.

Strengths:

- Very fast.

- Simple to implement.
- Useful when immediate response is important.

Weaknesses:

- Not guaranteed to find the minimum-cost route.
- Sensitive to heuristic errors.
- Straight-line distance does not represent traffic or risk.

ii. A* Search

A* combines actual travel cost and heuristic estimate:

$$f(n) = g(n) + h(n)$$

where $g(n)$ is the actual cost from the ambulance station to the current node, $h(n)$ is the estimated remaining cost, and $f(n)$ is the estimated total route cost.

A* therefore considers both the route already travelled and the estimated remaining distance. With an admissible heuristic in a static environment, A* can find an optimal route.

iii. Impact of heuristic accuracy

Greedy Best-First Search depends strongly on $h(n)$. An accurate heuristic can make it fast, while a poor heuristic may cause a very bad route.

A* is generally more robust because it considers $g(n) + h(n)$. An accurate heuristic reduces the number of nodes searched. A weak heuristic makes A* approach Uniform Cost Search and increases computation.

For emergency routing, straight-line distance should be supplemented with traffic, travel time, and risk information where possible.

iv. Dynamic road closures

Sudden road closures can invalidate previously generated routes.

Traffic costs can also change during the search.

Consequences include:

1. Previously generated nodes may become invalid.
2. Search may need to restart or repair the route.
3. Travel costs can change while searching.
4. A previously optimal route may become unsafe.
5. Continuous updates increase computation.

The system should therefore use dynamic replanning rather than relying on a single static search result.

v. Recommended algorithm

A* with dynamic replanning is the most suitable basic approach.

A useful route cost is:

Cost = Travel Time + Traffic Penalty + Risk Penalty

A* provides a better balance between speed and optimality than Greedy Best-First Search, while dynamic replanning allows the system to respond to closures and changing congestion.

Recommended architecture:

A* + real-time traffic information + dynamic replanning + safety constraints.

This balances real-time response, route quality, and ambulance safety.

3. Airline Crew Scheduling Optimization

Problem formulation

Crew scheduling is a large-scale constrained optimization problem.

Let:

$x_{ij} = 1$ if crew member i is assigned to flight j

$x_{ij} = 0$ otherwise

A possible objective is:

Minimize $Z = w_1(\text{Crew Cost}) + w_2(\text{Layover Time}) + w_3(\text{Scheduling Conflicts}) + w_4(\text{Delay Penalties})$

Subject to:

- Every flight has sufficient crew.
- A crew member cannot be assigned to overlapping flights.
- Required rest periods are maintained.
- Maximum working hours are respected.
- Crew qualifications match flight requirements.
- Base and airport constraints are satisfied.
- Regulatory requirements are followed.

The search space is enormous because thousands of crew members may be assigned across hundreds of flights. Exhaustive traditional search is therefore impractical.

i. Hill Climbing

Hill Climbing starts with an initial crew schedule and repeatedly moves to a neighboring schedule with a better objective value.

Possible neighborhood operations include:

- Swapping two crew members.
- Moving a crew member to another flight.
- Changing a layover.
- Replacing an assignment.
- Adjusting a flight sequence.

The algorithm stops when no neighboring solution improves the objective.

Its main limitation is that it only considers local improvements and can therefore become trapped before reaching the global optimum.

ii. Local maxima, plateaus, and ridges

Local maximum: A schedule may be better than all immediate neighboring schedules but worse than a distant schedule. For example, reaching a much better crew schedule may require temporary increases in cost.

Plateau: Many neighboring schedules have nearly identical evaluation values. The algorithm may not know which direction to choose and can stop or waste computation.

Ridge: Improvement may require several coordinated changes rather than one change. For example, changing one crew assignment may violate rest constraints, while changing three connected assignments together may produce a much better legal schedule.

iii. Simulated Annealing and Genetic Algorithms

Simulated Annealing can occasionally accept a worse solution, allowing it to escape local optima. The probability of accepting a worse solution can be represented as:

$$P = e^{(-\Delta E/T)}$$

At high temperature, exploration is greater. As temperature decreases, the algorithm focuses on refinement.

A Genetic Algorithm maintains a population of possible crew schedules and repeatedly performs:

Initial population → Fitness evaluation → Selection →
Crossover → Mutation → New population.

A chromosome can represent crew assignments. Fitness can penalize crew cost, layover cost, conflicts, delays, and regulatory violations.

Genetic Algorithms can explore many solutions simultaneously, escape local optima, handle complex constraints, and adapt to changing schedules.

iv. Recommended approach

A hybrid Genetic Algorithm with local improvement and constraint handling is recommended.

The Genetic Algorithm provides broad exploration and reduces the risk of local optima. Local search then improves promising schedules efficiently.

A practical architecture is:

Flight Schedule + Crew Data

- Constraint Generation
- Initial Feasible Population
- Genetic Algorithm
- Selection → Crossover → Mutation
- Constraint Repair
- Local Search Improvement
- Best Feasible Crew Schedule
- Flight Delay Detected
- Re-optimization

For real-time delays, unaffected assignments can be preserved while only the disrupted portion is re-optimized. This improves scalability and adaptability.

*This order assumes the graph supplied with your question is the simple sequence (S-A-B-C-D-E-G).

Important Point

If the actual displayed graph contains alternative branches, Greedy and A* can produce different expansion orders and

possibly different final costs. Greedy can choose a path with a small heuristic value but a large actual cost, whereas A* considers both $(g(n))$ and $(h(n))$.

Final Summary

[
 $\boxed{\text{Greedy Best-First Search uses only } h(n)}$

]
[
 $\boxed{\text{A* Search uses } g(n)+h(n)}$

] Therefore, A* is generally more reliable for finding the least-cost path, while Greedy Best-First Search may reach a goal faster but does not guarantee an optimal path.